

Optimal Athletic Programming in Triathlon using Linear and Integer Programming

Emmanouil Vichos (AM 1100830)

Linear and Combinatorial Optimization — University of Patras

Supervisor: Prof. S. Daskalaki

September 20, 2026

Abstract

Triathlon combines swimming, cycling and running, and its triple nature generates three interconnected decision problems: how to allocate training load, which races to enter, and how to pace a race. This project models all three with Linear Programming (LP) and 0–1 Mixed-Integer Programming (MIP), using fifteen months of the author’s real training data (574 activities from a Strava export). The models form a pipeline: the fitness state produced by the training model gates race selection and sets the energy budget for pacing. Beyond the base solutions, the project exercises the full course syllabus — Simplex-based LP solving, duality and shadow-price interpretation, right-hand-side and objective-coefficient ranging, an explicit primal–dual pair, Branch & Bound with LP relaxation gaps, and a network-flow reduction with an empirical total-unimodularity demonstration. Several optimal solutions reproduce established coaching wisdom (build-then-taper periodization, “steady bike, hard run” pacing) without it being imposed, while sensitivity analysis reveals which recommendations are driven by the objective and which by the constraints. A final integration step shows that the three models are coherent only when the race-day energy budget depends on form rather than fitness alone — which makes the training objective derived rather than assumed — and resolves the circular dependency between training plan and race calendar by iterating to a fixed point.

1 Introduction and Motivation

A competitive age-group triathlete faces three planning problems at different time scales. Over a macrocycle of 12–20 weeks they must decide how many hours to train in each sport and at which intensity, trading fitness gains against accumulated fatigue. Over a season they must choose a race calendar under a money budget, limited days off work, and physiological recovery requirements. Finally, on race day they must distribute a limited energy reserve across three consecutive legs to minimize finishing time.

These problems are usually solved by rules of thumb. This project instead treats them as mathematical programs and asks what an optimizer would do: Model 1 is an LP that allocates weekly training load; Model 2 is a 0–1 MIP that selects the race calendar; Model 3 is an LP that computes a pacing strategy. The three are coupled through the athlete’s fitness state: Model 1’s terminal CTL/ATL gates race readiness in Model 2 and sets the energy budget in Model 3 (Fig. 1). The coupling is genuinely circular — the chosen A-race determines the macrocycle Model 1 should plan — and Section 9 resolves it by iterating to a fixed point rather than by fixing an order, and shows what the three objectives must satisfy to be mutually consistent.

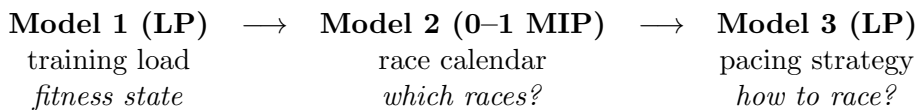


Figure 1: The three-model pipeline. Arrows carry the CTL/ATL fitness state; the A-race chosen by Model 2 feeds back into Model 1’s macrocycle (Section 9).

All three models are implemented in Python with PuLP as the modelling layer and CBC as the solver. The solution procedure, the code itself and the resulting console output are documented in the appendices: Appendix A lists the PuLP formulation of each model as extracted from the source files, Appendix B reproduces the verbatim solver output, and Appendix C maps every figure and table in this report to the module that generates it. The entire set of results is reproducible with one command, `python -m src.run_all`.

Two features distinguish the project from a textbook exercise. First, all inputs of Model 1 are estimated from the author’s real training history; the numbers below are personal, not stylized. Second, each model is accompanied by the analysis the course emphasizes: duals and their physiological meaning, value functions under RHS variation, integrality gaps, and alternative optima.

2 Literature

The training model rests on the fitness–fatigue framework of Banister et al. [1], applied to triathletes by Millet et al. [2] and criticised by Hellard et al. [3]; taper practice follows Mujika [4], with a recent optimization treatment in Ceddia et al. [5]. Race selection is a knapsack-type selection problem in the integer-programming modelling tradition [6, 7]; sports scheduling as an IP application area is surveyed by Ribeiro [8] and advocated pedagogically by Trick [9]. Pacing is reviewed by Abbiss and Laursen [10]; the cycling–running asymmetries behind our coupling constraint are documented by Millet et al. [11] and triathlon-specific factors by Wu et al. [12], with a numerical-optimization approach in Ni et al. [13]. General LP/OR background follows Dantzig [14] and Hillier–Lieberman [15].

3 Data

The data pipeline processes the author’s Strava bulk export: 598 activities from October 2023 to June 2026, of which 574 swim/bike/run sessions are kept. Each activity contributes date, sport, moving time, average heart rate (HR) and, for cycling, average power. For every activity a Banister training impulse is computed,

$$\text{TRIMP} = t \cdot \text{HRr} \cdot 0.64 e^{1.92 \text{HRr}}, \quad \text{HRr} = \frac{\text{HR} - \text{HR}_{\text{rest}}}{\text{HR}_{\text{max}} - \text{HR}_{\text{rest}}},$$

with t in minutes. The 125 activities without HR (mostly older runs) receive a sport-specific default intensity. Daily loads are smoothed exponentially into a chronic training load (CTL, $\tau = 42$ days) and an acute training load (ATL, $\tau = 7$ days); the balance TSB = CTL – ATL measures freshness. At the time of the study the athlete’s state was $\text{CTL}_0 = 78.1$, $\text{ATL}_0 = 91.5$, $\text{TSB} = -13.4$: solid fitness, currently fatigued.

Each activity is also classified into an intensity bucket — easy (Z1–2), moderate (Z3), hard (Z4+) — relative to a *sport-specific* lactate-threshold HR (run 173, bike 166, swim 164 bpm), since HR reads systematically lower in cycling and swimming at equal effort. The TRIMP-per-hour rates r_{db} by (sport, bucket) that parameterize Model 1 are estimated from the data where at least five activities exist and from the Banister formula at the bucket’s representative HR otherwise (4 of 9 cells, flagged as theoretical).

TRIMP/hour	easy (Z1–2)	moderate (Z3)	hard (Z4+)
swim	43.5	94.1*	136.6*
bike	68.3	86.9	142.7*
run	81.6	102.8	166.0*

Table 1: Estimated load rates r_{db} . Starred cells have fewer than five observed activities and use the theoretical Banister rate at the bucket’s representative heart rate.

Candidate races for Model 2 are a constructed but documented dataset of 17 events (Greek domestic, European, and long-haul; entry fees at official price ranges, travel costs and vacation-day requirements by destination), with deliberate structural features: same-weekend conflicts, a

championship pair two weeks apart, and a high-value early-season race the athlete cannot be fit for. The anchor events were verified against real 2025/2026 calendars (e.g. Ironman 70.3 Kraichgau and Nice, WTCS Hamburg, London T100, Chicago Triathlon, Epidavros Legacy, the Hellenic Championship on the federation’s published calendar); slots without a verifiable counterpart are explicitly labeled representative. All assumptions and per-race verification statuses are recorded in the repository (`docs/race_data.assumptions.md`).

4 Solution Method

All three models are written in Python with PuLP as the algebraic modelling layer and solved with CBC, the toolchain used in the course. Each model is built by a `build_model` function that returns the `pulp.LpProblem` together with its variable dictionaries; a `solve` wrapper calls `PULP_CBC_CMD` and returns the status string, and an `extract_solution` function converts the solved problem into data frames of the plan, the trajectory and the dual values. Keeping these three steps separate is what allows every sensitivity experiment in Sections 5.3 and 8 to rebuild and re-solve the same model under perturbed data without duplicating any formulation.

Models 1 and 3 are pure linear programs and CBC solves them with the simplex method; Appendix D works a two-variable reduction of Model 3 by hand, through Phase I and a single Phase II pivot, to make the algorithm explicit rather than implicit. Model 2 is a 0–1 program and is solved by branch and bound on the LP relaxation; the relaxation is also solved on its own so that the integrality gap and the fractional variables can be inspected, and clique cuts are added separately to show how a tighter formulation closes part of that gap. Because every logical condition in Model 2 is expressed through standard linear templates rather than big- M terms, the relaxation is tight enough that branch and bound terminates immediately at this size.

Model	Type	Variables	Constraints	Solve time	Objective
Model 1 (training)	LP	192	209	5 ms	−11.44
Model 2 (calendar)	0–1 MIP	17	27	5 ms	571.67
Model 3 (pacing)	LP	15	7	1 ms	140.46

Table 2: Problem sizes and CBC solve times for the baseline instances (`results/tables/model_sizes.csv`).

The solved output of all three models is reproduced verbatim in Appendix B, the code that produces it in Appendix A, and the complete set of results can be regenerated with the single command `python -m src.run.all`.

5 Model 1 — Optimal Training-Load Allocation (LP)

5.1 Formulation

For disciplines $D = \{\text{swim, bike, run}\}$, buckets $B = \{\text{easy, mod, hard}\}$ and weeks $W = \{1, \dots, T\}$ ($T = 16$ for an Olympic-distance goal), the decision variables are training hours $h_{dbw} \geq 0$ ($3 \times 3 \times 16 = 144$ variables). Weekly load and the Banister recursion enter as equality-defined auxiliaries, keeping the model an LP:

$$\begin{aligned}
L_w &= \sum_{d,b} r_{db} h_{dbw}, \\
\text{CTL}_w &= \lambda_c \text{CTL}_{w-1} + (1 - \lambda_c) \frac{L_w}{7}, & \lambda_c &= e^{-7/42}, \\
\text{ATL}_w &= \lambda_a \text{ATL}_{w-1} + (1 - \lambda_a) \frac{L_w}{7}, & \lambda_a &= e^{-7/7}.
\end{aligned}$$

The objective maximizes predicted race-day performance in the Banister sense,

$$\max p_T = k_1 \text{CTL}_T - k_2 \text{ATL}_T, \quad (k_1, k_2) = (1, 2).$$

Banister units are arbitrary: the *level* of p_T (here negative, since $k_2 > k_1$) has no physical meaning — only differences between plans and the marginal values (duals) are interpretable. Maximizing TSB

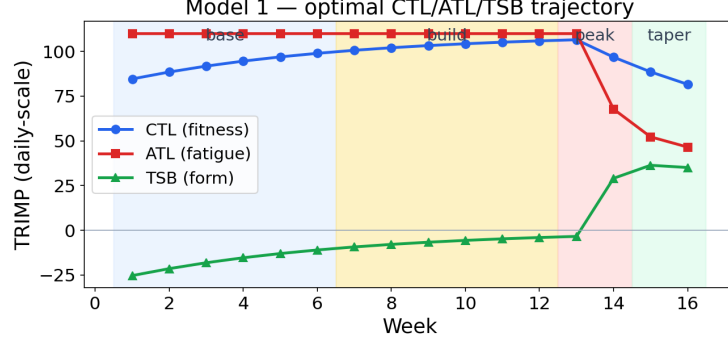


Figure 2: Optimal CTL/ATL/TSB trajectory. The plan rides the fatigue ceiling (ATL = 110) for 13 weeks, building CTL from 78 to 107, then tapers.

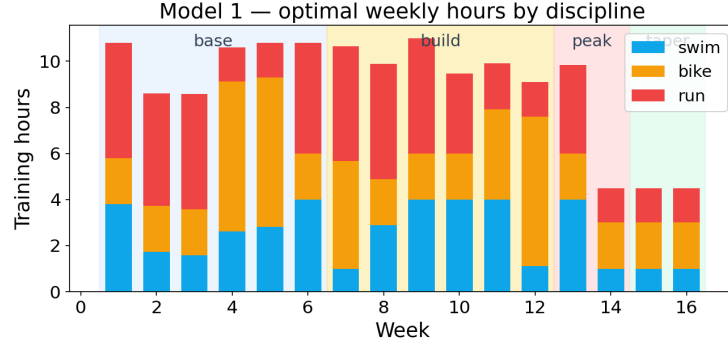


Figure 3: Optimal weekly training hours by discipline across macrocycle phases.

alone would be degenerate (the optimum would be to rest); the Banister form rewards early CTL accumulation, whose contribution persists, while early-week ATL decays away by race day. The constraints, for every week w :

$$\sum_{d,b} h_{dbw} \leq H_w^{\max} \quad \text{phase-dependent hours ceiling (12 h baseline)} \quad (1)$$

$$h_d^{\min} \leq \sum_b h_{dbw} \leq h_d^{\max} \quad \forall d \quad \text{discipline bounds (pool access, injury risk)} \quad (2)$$

$$\sum_d (h_{d,\text{mod},w} + h_{d,\text{hard},w}) \leq 0.2 \sum_{d,b} h_{dbw} \quad \text{80/20 polarized-training rule} \quad (3)$$

$$\text{ATL}_w \leq 110 \quad \text{overtraining guard} \quad (4)$$

$$L_w \leq 1.1 L_{w-1} \quad \text{ramp-rate limit } (w \geq 2) \quad (5)$$

$$L_w \leq \theta_w \bar{L}_{\text{build}} \quad w \in \text{taper} \quad \theta_{15} = 0.6, \theta_{16} = 0.4 \quad (6)$$

where $\bar{L}_{\text{build}}$ is the (linear) average build-phase load. The solved LP has 192 variables and 209 constraints.

5.2 Results

The optimal plan (Fig. 2) holds weekly load exactly at the value that keeps ATL on its ceiling for 13 weeks — the fatigue constraint, not available time, is binding — and then cuts volume to one third for the final three weeks, arriving at race day with CTL = 81.6, ATL = 46.5, TSB = +35. Notably, only weeks 15–16 are *constrained* to taper; the optimizer starts tapering in week 14 by choice. Build-then-peak-then-taper periodization is thus a *property of the optimum*, not an assumption. The weekly discipline allocation (Fig. 3) keeps all three sports within realistic bounds while the intensity mix sits exactly on the 80/20 polarization constraint — the coaching rule is recovered as a binding constraint of the optimal plan.

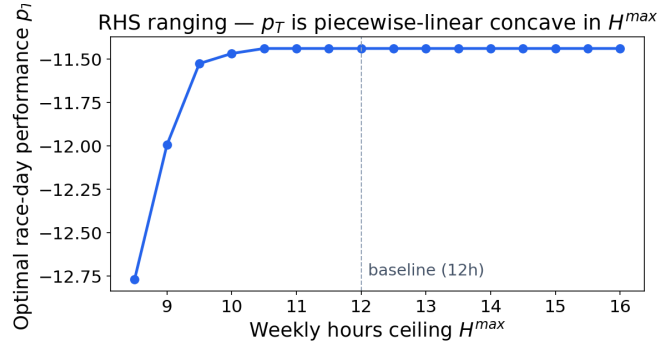


Figure 4: Value function of the weekly-hours RHS: piecewise-linear, concave, saturating at 10.5 h/week.

5.3 Duality and sensitivity

RHS ranging. Re-solving for hour ceilings $H^{\max} \in [8, 16]$ traces the LP value function (Fig. 4): empirically piecewise-linear and concave, with slopes $1.55 \rightarrow 0.94 \rightarrow 0.12 \rightarrow 0.06 \rightarrow 0$. Below 8.5 h the model is infeasible (discipline minimums collide with taper caps); above 10.5 h extra hours are worthless because the ATL ceiling binds first. For this athlete, recovery capacity — not free time — limits performance.

Shadow prices. The largest duals in absolute value belong to the race-week discipline minimums: a forced hour of running in week 16 costs 12.9 performance points — the model would rather rest completely. The ATL-ceiling duals peak in late build (week 12: +0.07 per TRIMP/day of extra fatigue tolerance), quantifying the value of recovery capacity when it matters most.

Alternative optima. Fixing the objective at p^* and re-solving with total season hours as a secondary objective yields plans from 124.2 h to 162.3 h at *identical* performance: the optimal face has positive dimension, and the same race-day form is achievable with 38 fewer training hours. Practically, the efficient (minimum-hours) plan is preferable; theoretically, this is LP degeneracy made visible. Realistic per-discipline weekly caps were added to bound this degeneracy for the reported plan.

Benchmark against a coaching heuristic. A standard “fill the available hours, 20/45/35 discipline split, 80/15/5 intensity split” plan violates the ATL ceiling in 13 of 16 weeks and scores $p_T = -69.5$ versus the optimum -11.4 . The heuristic does, however, arrive at race day with TSB = +21, inside the band +15...+25 recommended by coaching practice, while the LP over-tapers to +35 — see the k_2 discussion in Section 8.

5.4 Validation against actual training

Parameter estimation from history is not validation, so the optimal plan was compared against what the athlete *actually did* in the final 16 weeks of the export: 6.6 h/week at a 37% moderate+hard share, versus the optimum’s 9.0 h/week at 15% (Fig. 5). The athlete trains less and substantially *harder* than optimal — the exact anti-polarized pattern the endurance literature warns against — and the model quantifies the cost: the actual pattern cannot sustain the CTL trajectory the optimal plan achieves within the same fatigue ceiling. This turns the LP from an abstract exercise into a personally actionable diagnosis.

Duality theory itself was also verified numerically on the solved models: the primal optimum equals the dual objective $\sum_i y_i b_i$ (strong duality) and $y_i \cdot \text{slack}_i = 0$ holds for every constraint (complementary slackness), on both Model 1 and Model 3, to 10^{-6} .

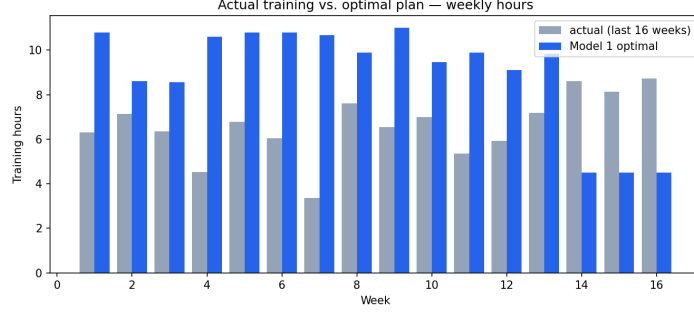


Figure 5: Actual weekly training (grey) vs. the optimal plan (blue) over 16 weeks.

6 Model 2 — Race-Calendar Selection (0–1 MIP)

6.1 Formulation

For candidate races $i \in R$ with week t_i and distance class c_i , binary variables x_i select the season calendar. The risk-adjusted value of a race is

$$v_i = q_i (\alpha \text{pts}_i + \beta \text{strat}_i + \gamma \text{enj}_i + \varphi \text{fit}_i),$$

where q_i is the probability of good race-day weather, enjoyment enj_i aggregates subjective organization/scenery/swim scores, and suitability fit_i is computed from *objective* course data — elevation gain per km per leg matched to the athlete’s flat-course preference. The program is a two-dimensional knapsack with temporal side constraints:

$$\max \sum_i v_i x_i \quad \text{s.t.} \quad \sum_i \kappa_i x_i \leq 2500 \text{ EUR}, \quad \sum_i \delta_i x_i \leq 12 \text{ days},$$

at most two races per month; recovery spacing $x_i + x_j \leq 1$ for pairs closer than the recovery requirement of the earlier race (sprint 1, Olympic 2, 70.3 4 weeks); at least one championship-quality “A-race”; and a readiness gate from Model 1 — a projected season CTL curve (off-season start, load ramping to Model 1’s build level) must reach a class-specific requirement by race week, which excludes the high-value but too-early Dubai 70.3.

Each side constraint instantiates a standard integer-modeling template. The pairwise recovery rows $x_i + x_j \leq 1$ are the “at most one of two” (either–or) pattern; the clique inequalities added below generalize it to “at most one of k ”; the A-race row $\sum_{i \in A} x_i \geq 1$ is the “at least one of N ” (set-covering) pattern; the monthly caps are “at most M of N ”; and the readiness gate is variable fixing, $x_i = 0$, derived from Model 1 rather than assumed. Keeping every logical condition in one of these linear forms is what allows the model to remain a pure 0–1 program solvable by Branch & Bound, with no big- M terms and hence no weak-relaxation penalty.

6.2 Results

The optimum selects 8 of 17 races for a value of 571.7 (Table 3, Fig. 6), spending 2250 of 2500 EUR and 11 of 12 vacation days — both knapsack dimensions are nearly binding. The recovery constraint forces a choice between the National Championship and the home-soil 70.3 two weeks later; the optimizer picks the 70.3. The LP relaxation scores 600.5, an integrality gap of 5.0%, with eight fractional variables including the championship conflict split exactly 0.5/0.5 — a textbook illustration of why Branch & Bound is needed. At the relaxation’s optimum the budget dual is 0.094 value/EUR while the vacation-days dual is ≈ 0 : at the baseline, money, not time off, is the scarce resource.

Budget sweep. Varying the money budget from 500 to 4000 EUR (Fig. 7) shows the characteristic IP behavior: the optimal value increases in *steps* (races are indivisible) beneath the concave LP-relaxation curve. Above 3000 EUR the value saturates at 592.7 because all 12 vacation days are consumed — the binding constraint switches dimension.

	Race	Date	Class	Cost (EUR)	Value
R01	Attica Sprint (season opener)	Mar 14	sprint	45	45.1
R02	TRIMORE Rethymno	Apr 18	olympic	290	83.8
R03	TRIMORE Syros	May 9	olympic	210	56.6
R05	Epidavros Legacy Standard	Jun 6	olympic	60	69.1
R07	Thessaloniki Sprint	Jun 27	sprint	160	46.6
R09	Nafplio	Jul 18	olympic	135	48.0
R11	Ironman 70.3 Budapest	Aug 22	70.3	770	104.7
R14	Ironman 70.3 Costa Navarino	Sep 26	70.3	580	117.7

Table 3: Optimal 2027 race calendar (A-race: Costa Navarino).

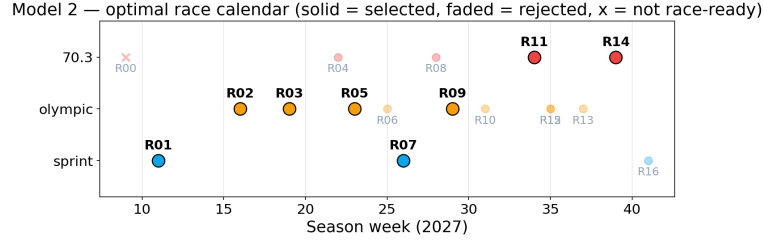


Figure 6: Season timeline: selected races (solid), rejected (faded), not race-ready (x).

Strengthening the formulation: clique cuts. The recovery conflicts are encoded pairwise ($x_i + x_j \leq 1$), which is integer-equivalent but weak in relaxation: three mutually conflicting races at $x = \frac{1}{2}$ satisfy all pairwise constraints. Enumerating the maximal cliques of the conflict graph (Bron–Kerbosch; 17 nodes) yields four valid inequalities $\sum_{i \in C} x_i \leq 1$ with $|C| \geq 3$ (largest clique: four August–September races). Adding them tightens the LP relaxation from 600.5 to 591.0, closing a third of the integrality gap ($5.04\% \rightarrow 3.38\%$) without touching the integer optimum — a small demonstration of the cutting-planes idea underlying modern branch-and-cut solvers.

6.3 The dual problem and its interpretation

The LP relaxation of Model 2 is already in the standard primal form $\max\{c^T x \mid Ax \leq b, x \geq 0\}$, so its dual follows immediately as $\min\{b^T y \mid A^T y \geq c, y \geq 0\}$. Writing it out with one dual variable per constraint family — y_B (money budget), y_D (vacation days), y_m (races in month m), y_{ij} (recovery conflict between races i and j), y_A (A-race requirement) — gives

$$\min \mathcal{B} y_B + \mathcal{D} y_D + \sum_m M y_m + \sum_{(i,j)} y_{ij} - y_A$$

subject to, for every candidate race i ,

$$\underbrace{\kappa_i y_B + \delta_i y_D + y_{\text{month}(i)} + \sum_{j: (i,j) \in C} y_{ij} - [i \in A] y_A}_{\text{shadow cost of the resources race } i \text{ consumes}} \geq v_i, \quad y \geq 0.$$

The dual constraint is the economic statement of the problem: a race enters the optimal calendar only when the value it delivers covers the shadow cost of everything it consumes — euros, days off, its slot in the month, and the races it blocks through recovery. By complementary slackness, races selected strictly inside the budget satisfy this with equality, while rejected races have strict surplus, and that surplus is exactly the amount by which their value would have to rise to make them worth entering (Section 8).

Solving the relaxation gives 12 nonzero duals out of 27 constraints, and their ranking is the project’s most counter-intuitive result (Table 4). The money budget prices at only 0.094 per euro and the vacation-day budget at ≈ 0 , while the *recovery* conflicts dominate: the constraint separating the Hellenic Championship from the Costa Navarino 70.3 two weeks later carries a shadow price of 56.4, by far the largest in the model. For this athlete the binding resource is neither money nor time

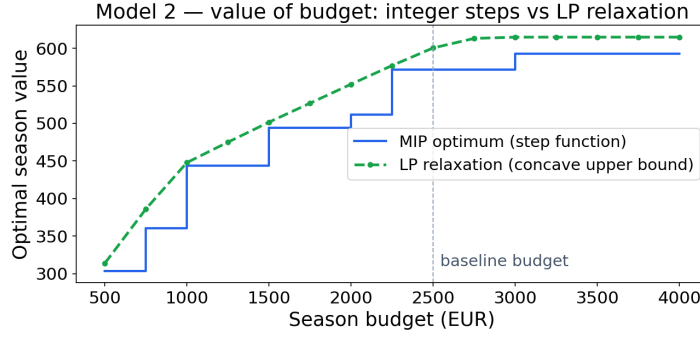


Figure 7: Season value vs. budget: the MIP optimum is a step function; the LP relaxation is a concave upper bound. Beyond 3000 EUR the vacation-days budget becomes binding and money stops mattering.

off work but *calendar space*: the physiological requirement to recover between races is what truly constrains the season.

Dual variable	Meaning	Shadow price
$y_{13,14}$	Hellenic Champs vs. Costa Navarino 70.3 conflict	56.40
$y_{06,07}$	Hamburg vs. Thessaloniki conflict	31.67
$y_{11,13}$	Budapest vs. Hellenic Champs conflict	26.77
$y_{\text{ready},00}$	fitness-readiness gate on Dubai 70.3	25.59
y_B	one extra euro of season budget	0.094
y_D	one extra vacation day	≈ 0

Table 4: Largest dual values of Model 2’s LP relaxation. Recovery spacing, not budget, is the scarce resource.

6.4 Network-flow extension

Dropping the two knapsacks, the monthly cap and the A-race constraint leaves only recovery spacing, and the problem becomes a *longest path in a DAG*: nodes are race-ready races in week order, an arc $i \rightarrow j$ exists when the gap respects the recovery of race i , and node weights are the v_i . Recovery requirements compose along chains, so feasible calendars and S – T paths coincide. Three methods were implemented and agree exactly ($z = 639.4$): dynamic programming in topological order; a flow LP with *continuous* variables; and the restricted MIP. The flow LP returns an integral solution without any integrality constraints — its constraint matrix is a network matrix, hence totally unimodular. The knapsack constraints are therefore precisely what breaks the network structure and forces Branch & Bound in the full model.

The flow formulation is also what makes the *Network Simplex Method* applicable. Its constraint matrix is the node–arc incidence matrix of the DAG (one +1 and one –1 per arc column), so every basis corresponds to a spanning tree of the network and a pivot is an arc exchange — entering arc in, leaving arc out — performed by updating tree potentials rather than by factorizing a general basis matrix. Because the basis is always integral, the method terminates at an integral flow without any branching, which is the algorithmic counterpart of the total unimodularity verified above. The present instance (18 nodes — 16 race-ready races plus source and sink — and 136 arcs) is far too small for the speed advantage to matter, so it is solved with CBC’s general simplex; the point is structural, namely that the relaxed race-selection problem belongs to the class of problems Network Simplex is designed for, and leaves that class exactly when the two knapsack rows are added.

7 Model 3 — Race Pacing Strategy (LP)

7.1 Formulation

For legs Λ and intensity zones $Z1$ – $Z5$, the variables $t_{\ell z} \geq 0$ are minutes in zone z during leg ℓ . Within a zone, speed and metabolic rate are constants, linearizing the nonlinear speed–energy curve: bike speed scales with the cube root of power (aerodynamic drag), $v_z = \phi(\gamma) v_{\text{FTP}} (P_z / \text{FTP})^{1/3}$ with

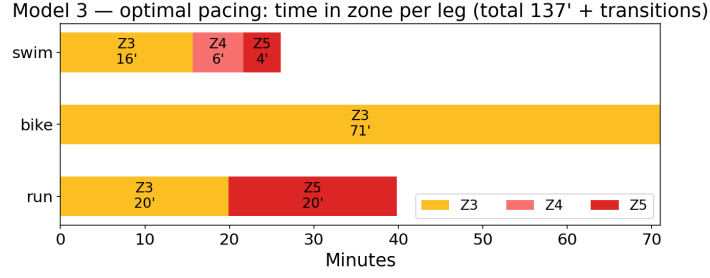


Figure 8: Optimal pacing for the Olympic distance: steady Z3 bike, hard run. Total 140.4 min including transitions; energy budget exactly exhausted.

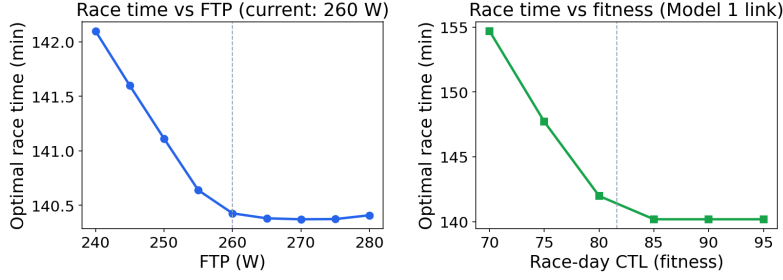


Figure 9: Parametric analysis of Model 3. Left: race time vs. FTP — watts are worth ~ 6 s each below 260 W and almost nothing above. Right: race time vs. race-day CTL — fitness is the binding resource up to CTL ≈ 85 .

FTP = 260 W. The factor $\phi(\gamma) \leq 1$ accounts for the course’s climbing γ (metres per km): at equal power a rider covers less ground on a hilly course, because the time lost climbing exceeds the time regained descending. It is obtained from the cycling power balance $P = v(kv^2 + F_{\text{roll}} + mgs)$ by solving for the up- and downhill speeds at constant power and taking their harmonic mean over equal distances, with k back-solved from the athlete’s own flat reference speed — so nothing in ϕ is fitted to race results. It is convex in γ ($\phi = 0.97$ at 9 m/km, 0.91 at 15), so a linear penalty would be only a local approximation. The baseline instance is a flat reference course, $\gamma = 0$, $\phi = 1$. Swim/run speeds are fractions of threshold pace; energy cost per km rises with intensity (economy factors 0.95–1.20). The program minimizes total time subject to covering each leg’s distance, an overall energy budget $E^{\text{tot}} = k_E \cdot \text{CTL}_T$ linked to Model 1, per-leg caps on time at Z4–Z5, and a bike→run coupling: each kJ of hard bike work reduces effective run coverage by φ km.

7.2 Results

The optimal Olympic-distance race takes 140.4 minutes: swim 26.1 (mixed Z3–Z5 at the intensity cap), bike 71.0 entirely at Z3 (33.8 km/h), run 39.8 (half Z3, half Z5 at the cap), with the 9389 kJ energy budget exactly binding (Fig. 8). The optimizer *derives* the standard triathlon advice — ride steady, run hard: hard biking buys little speed (cube-root curve) while draining both the shared energy budget and run capacity through the coupling term.

7.3 Duality: what is a watt worth?

The duals answer athlete questions directly. The energy dual is -0.0071 min/kJ: one extra kJ of aerobic capacity saves 0.43 s. The distance duals price each km: an extra swim km would cost 26.4 min, bike 2.5, run 6.3. Parametric re-solving over FTP (Fig. 9, left) shows each watt below the current 260 W is worth ≈ 6 s, but the marginal value collapses at 260–270 W: with a fixed energy budget, more power cannot be fueled. The CTL sweep (right) confirms the flip side: one CTL point is worth 69–84 s up to CTL ≈ 85 , then saturates when speed caps bind instead. For this athlete the data-driven advice is unambiguous — build endurance, not peak power.

7.4 Validation against real races

The author raced the *Epidavros Standard Triathlon* (Olympic distance) on 7 September 2025, finishing in 2:43:26. The training data pipeline reconstructs the fitness state on that day: CTL = 55.6, TSB = −14 — far below the model’s assumed optimal preparation (CTL = 81.6, TSB = +35), so the prediction and the result are compared leg by leg rather than in total (Table 5).

Leg	Model (optimal prep)	Actual (Epidavros)	Assessment
Swim	26.3 min	25.8 min	within 0.5 min — threshold pace validated
Bike	78.9 min (29.4 km/h)	91.5 min (25.4 km/h)	flat-course assumption fails on hills
Run	37.7 min	42.0 min	−10% — optimistic under a fatigue deficit
Transitions	3.5 min	4.2 min	reasonable
Total	146.4 min	163.4 min	

Table 5: Model 3 (flat baseline, $k_E = 142$) vs. the actual race, 7 Sep 2025.

The comparison separates parameter quality cleanly. Where zone speeds derive from *personally measured* thresholds (swim pace, run pace), predictions land within minutes. The 12.6-minute bike gap is a course-model mismatch: the 35 km/h-at-FTP assumption describes a flat time-trial course, while the Epidavros bike leg is hilly — a course-specific speed curve (or the elevation data already used in Model 2) would close most of it. The race also *calibrates* the energy link: at the actual race-day CTL of 55.6 the a priori budget $E = 115 \cdot \text{CTL}$ is *infeasible* — the model says the race cannot be finished — yet the athlete finished it. Raising k_E to ≈ 142 is the smallest change that restores feasibility, which is how that constant is calibrated here; it does not reproduce the finishing time (the model still predicts 146.4 min against 163.4 actual, the remainder being the hills and the within-leg fatigue the formulation omits). A single real race thus bounds the model’s least-measurable constant from below, and all energy-related duals scale by $\approx 142/115$ under the recalibration.

A second, more recent race probes a different problem size: the *Spetsathlon 2026* (17 May 2026; non-standard sprint — 750 m swim, 25 km hilly bike, 4.75 km run), finished in 1:24:28 at a reconstructed race-day state of CTL = 64.9, TSB ≈ 0 . Model 3 re-solved on this course geometry predicts 78.5 min vs. 84.5 actual (Table 6).

Leg	Model	Actual (Spetsathlon)	Assessment
Swim	12.9 min	13.2 min	within 0.3 min, again
Bike	43.3 min (33.8 km/h)	50.9 min (28.8 km/h)	flat-course assumption, again
Run	18.7 min	17.7 min	athlete <i>beat</i> the model’s Z5 speed cap

Table 6: Model 3 vs. the Spetsathlon 2026 sprint (transitions excluded).

Two findings sharpen the picture. First, the run prediction errs in the *opposite* direction from Epidavros: over only 18 minutes the athlete sustained 3:44/km, faster than the model’s Z5 ceiling of $1.05 \times \text{threshold}$ — the fixed zone grid underestimates the anaerobic contribution available in short races. Second, and theoretically most interesting, at sprint distance the energy budget is *slack* (the solution is identical for $k_E = 115$ and 142): the binding constraints switch from the energy budget (Olympic) to the per-leg speed and intensity caps (sprint). The LP’s active set changes with the problem’s “size” exactly as duality theory describes — and it matches physiology, where sprint racing is speed-limited and Olympic-distance racing is energy-limited.

The 70.3 and the fueling extension. The athlete’s *Ironman 70.3 Greece, Costa Navarino* (26 October 2025; 1.9 km / 90 km / 21.1 km, finish 5:39:05, race-day CTL = 71.4, TSB = +5.6 after a genuine taper) — incidentally the same event, in its 2027 edition, that Model 2 selects as the season’s A-race (R14) — exposed a structural omission: at 70.3 distance the model is *infeasible* even with the calibrated $k_E = 142$ — stored energy alone cannot cover a 5–6 hour race, and indeed no athlete attempts one unfed. In-race carbohydrate intake at r kJ/min enters the model linearly, as a reduction

of every zone’s *net* drain:

$$\sum_{\ell,z} (e_{\ell z} - r) t_{\ell z} \leq E^{\text{tot}},$$

so the extension costs nothing in tractability. Calibrating r against the actual race gives $r \approx 21$ kJ/min — which is ≈ 75 g of carbohydrate per hour, *precisely the standard sports-nutrition guideline*. With it, the model reproduces the swim within 0.4 min (33.3 vs. 32.9) and the bike within three minutes (177.5 vs. 180.3).

Fueling can further be made a *decision*: per-leg intake $F_\ell \geq 0$ (kJ) bounded by gut absorption, $F_\ell \leq \bar{r}_\ell \sum_z t_{\ell z}$ (swim 0, bike 25, run 15 kJ/min), with budget $\sum e_{\ell z} t_{\ell z} - \sum_\ell F_\ell \leq E^{\text{tot}}$ — still an LP. The optimizer then *reproduces nutrition practice*: nothing in the water, maximal intake on the bike (≈ 90 g/h), reduced intake on the run. The absorption duals price “gut training” directly: at 70.3, one extra kJ/min of bike absorption capacity is worth ≈ 1.7 minutes of race time — while at Olympic distance the same relaxation is worth almost nothing (speed caps bind, not energy). Electrolyte and hydration intake, though practically crucial, are deliberately not modeled: no defensible linear mechanism links them to speed, and the data cannot calibrate one — a scope boundary, not an oversight.

The run tells the most instructive story. The model predicts 86.3 min at even pacing; the athlete ran 115.2 with a dramatic positive split: km 1–10 at $\approx 4:53/\text{km}$, then “the wall” — km 11–21 at $\approx 5:52/\text{km}$ with heart rate *falling* from 162 to ≈ 145 bpm. Falling HR with collapsing pace is the signature of glycogen depletion, i.e. of the energy constraint the model encodes: the athlete spent the budget faster than the optimal (even) schedule and paid mid-race. The model cannot predict the wall’s *timing* (its zone speeds carry no within-leg fatigue dynamics — a documented limitation), but the mechanism that produced it is exactly the binding constraint of the formulation. Table 7 summarizes all three validations at the calibrated $k_E = 142$ and each race’s actual reconstructed fitness.

Race	Swim			Bike		Run	
	CTL	model	actual	model	actual	model	actual
Spetsathlon 2026 (sprint*)	64.9	12.9	13.2	47.3	50.9	18.7	17.7
Epidavros 2025 (Olympic)	55.6	27.2	25.8	88.8	91.5	38.9	42.0
IM 70.3 Costa Navarino 2025 (fueled)	71.4	33.3	32.9	184.6	180.3	86.4	115.2

Table 7: Model 3 vs. three real races (leg times, min) at $k_E = 142$ and actual race-day fitness. *non-standard course. Consistent misses: hilly-course bike speed (sprint, Olympic) and within-leg run fatigue (70.3).

What the bike residual is made of. The bike is the leg the model predicts least well, and its residual is informative rather than random. Had the course gradient been ignored — a naive flat-course speed curve — the total bike error across the three races would be 23.0 min and the model would be too fast on *every* course, the signature of a missing term. With $\phi(\gamma)$ in the formulation the error falls to 10.6 min and changes sign between races (Table 8): no remaining systematic bias.

Bike leg	m/km	no gradient	$\phi(\gamma)$	empirical fit	actual
Spetsathlon (24.4 km)	15.2	43.3	47.3	55.0	50.9
Epidavros (38.7 km)	15.5	78.9	88.8	87.7	91.5
Costa Navarino (88.5 km)	9.1	177.5	184.6	180.0	180.3
total absolute error		23.0 min	10.6 min	8.2 min	
parameters fitted to these races		0	0	2	

Table 8: The bike leg under three speed curves. The shipped model uses $\phi(\gamma)$, which fits nothing to these races. The empirical column is a two-parameter fit *on* them, and is therefore an upper bound on what such a correction could achieve rather than a competing result.

That last column is worth including because it bounds the error that remains. A linear penalty $c\gamma$, with c and k_E both re-fitted on these three races, reaches 8.2 min — only 2.4 min better than the

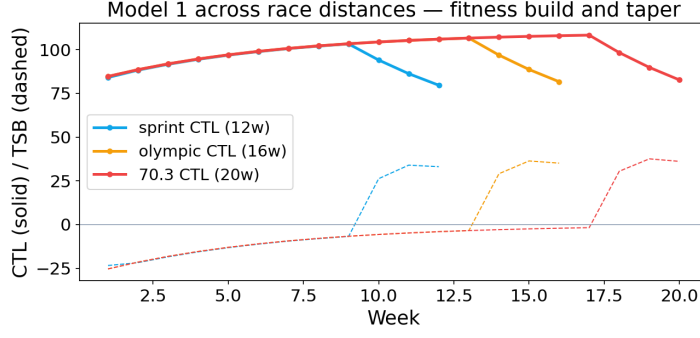


Figure 10: Model 1 across race distances: CTL build (solid) and TSB (dashed) for 12/16/20-week horizons. The build–taper structure is invariant.

derived ϕ , while spending two degrees of freedom on three observations, and with c moving from 0.014 to 0.008 if distances are taken as nominal rather than measured. Essentially all of the recoverable error is therefore attributable to gradient alone, and what is left is not something three races can identify. The sign of the residual is itself consistent with the model’s own simplification: ϕ assumes the rider holds threshold power on the descents, which no one does on technical courses, so the true penalty on the two hilly races should be a little larger than modelled — and the model is indeed still 3.6 and 2.7 min fast on exactly those two.

8 Cross-Model Sensitivity Analysis

Problem size (distance scenarios). Re-solving Model 1 for Sprint (12 weeks, 10 h ceiling), Olympic (16, 12 h) and 70.3 (20, 14 h) scales the LP from 144 to 240 variables. Longer horizons monotonically improve race-day performance (p_T : $-13.5 \rightarrow -11.4 \rightarrow -10.4$) — more weeks allow CTL to be built early and cheaply — while the optimal structure (ceiling-riding build, deep taper, TSB $+33 \dots +36$) is invariant (Fig. 10).

All instances solve in under 10 ms with CBC (Table 2).

Perturbing the cost vector c . Sensitivity analysis of the objective coefficients completes the picture begun by the RHS ranging of Section 5.3, and separates into the two standard cases.

Case A — coefficients of basic variables. In Model 1 the objective $p_T = k_1 \text{CTL}_T - k_2 \text{ATL}_T$ has exactly two nonzero cost coefficients, both of them hard to measure physiologically. Bisection outward from the baseline until the optimal plan changes gives the stability intervals $k_1 \in [0.68, 1.56]$ and $k_2 \in [1.29, 2.95]$: the entire optimal training plan — all 144 hour variables — is invariant to a $\pm 50\%$ misestimation of either Banister weight. The plan therefore does not rest on the least defensible numbers in the model.

Case B — coefficients of non-basic variables. In Model 2 the cost coefficients are the race values v_i , and selection plays the role of basis membership. For each race, bisection locates the critical perturbation at which the optimal calendar changes (Table 9). Selected domestic races (R01, R02, R03, R05, R07) can lose their *entire* value without leaving the calendar — they are cheap enough that the budget never rules them out — whereas the marginal picks (Nafplio, Budapest, Costa Navarino) sit only 9.95 points above expulsion. Symmetrically, the rejected races closest to entering (Nice 70.3, the Hellenic Championship, Valencia) need exactly $+9.95$: the matching thresholds reveal a near-tie in which one entering race displaces another, which is precisely the degeneracy that made the LP relaxation split those variables at $\frac{1}{2}$ in Section 6.2.

Fatigue weight k_2 . Sweeping $k_2 \in [1, 3]$ barely moves race-day TSB (35 ± 2): taper depth turns out to be pinned by the *constraints* (race-week discipline minimums), not by the objective. This is a reassuring robustness result — the plan does not hinge on an unmeasurable parameter — and it locates the earlier over-taper observation ($+35$ vs. the $+15 \dots +25$ coaching band) in the constraint set: relaxing race-week minimums, not retuning k_2 , would close that gap (Fig. 11).

Race	Role	Value	Allowed change
R01 Attica Sprint	selected (basic)	45.1	−45.11 (any)
R02 TRIMORE Rethymno	selected (basic)	83.8	−83.84 (any)
R09 Nafplio Olympic	selected (basic)	47.9	−9.95
R11 IM 70.3 Budapest	selected (basic)	104.7	−9.95
R14 IM 70.3 Costa Navarino	selected (basic)	117.7	−9.95
R08 IM 70.3 Nice	rejected (non-basic)	115.7	+9.95
R13 Hellenic Champs	rejected (non-basic)	94.7	+9.95
R16 MTRI Valencia	rejected (non-basic)	50.0	+9.95
R15 Chicago Triathlon	rejected (non-basic)	87.0	+121.00

Table 9: Objective-coefficient ranging in Model 2: how far each race’s value may move before the optimal calendar changes (full table in `results/tables/model2_cost_ranging.csv`).

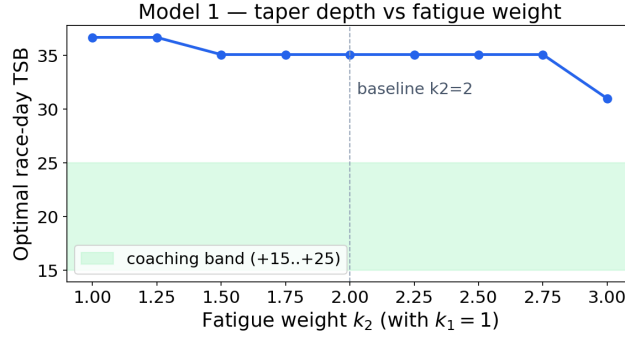


Figure 11: Race-day TSB vs. fatigue weight k_2 : nearly flat — taper depth is constraint-determined, robust to the objective’s calibration.

9 Integrating the Pipeline

Presenting three models in sequence raises two fair objections: that the models are only loosely coupled, and that the order of solution was chosen rather than justified. Both are addressed here.

9.1 Are the objectives coherent?

Model 1 maximizes a Banister performance index $p_T = k_1 \text{CTL}_T - k_2 \text{ATL}_T$, whereas the quantity the athlete actually cares about is the finishing time Model 3 computes. These need not agree — and by default they do not. Model 3’s energy budget $E = k_E \text{CTL}_T$ does not involve ATL at all, so as far as Model 3 is concerned the ideal preparation is maximal fitness at *any* level of fatigue, i.e. no taper. The disagreement is easy to miss while the models are solved separately, but it is stark once they are composed: pushing four materially different Model 1 plans through Model 3 returns *exactly the same* race time of 140.18 min for all four (Table 10). With a fitness-only budget the pacing model is simply blind to the training decision, and the “pipeline” does not really transmit anything.

Making the budget depend on race-day *form* repairs this. Let

$$E(\text{plan}) = E_{\text{base}} + k_E (\text{CTL}_T - \lambda \text{ATL}_T).$$

If $\lambda = k_2/k_1$ then $E = E_{\text{base}} + (k_E/k_1) p_T$, an increasing affine function of Model 1’s objective. Since Model 3’s optimal time is non-increasing in E (verified as a monotonicity test), it follows that

$$\arg \max p_T = \arg \max E = \arg \min \text{race time}.$$

Model 1’s objective is then not a proxy but a *derived* one, and the ratio k_2/k_1 ceases to be a free parameter: it is the fatigue sensitivity of the energy budget. Empirically the two rankings coincide exactly under $\lambda = k_2/k_1$ and fail to coincide under $\lambda = 0$ (Fig. 12).

There is one feasibility condition: $E > 0$ requires $\lambda < \text{CTL}_T/\text{ATL}_T$, which at the optimum is $81.64/46.54 = 1.75$. The value assumed throughout, $k_2 = 2.0$, therefore *cannot* be made coherent — it

CTL _T	ATL _T	coherent ($\lambda = 1.5$)		fitness-only ($\lambda = 0$)	
		p_T	race time		race time
91.95	57.55	5.62	163.68		140.18
88.99	52.25	10.61	142.14		140.18
81.64	46.54	11.83	140.18		140.18
75.42	44.44	8.77	147.64		140.18

Table 10: Four distinct training plans scored end-to-end. Under the coherent budget the plan with the largest p_T is the fastest; under the fitness-only budget every plan yields the same time, so Model 3 cannot see the training decision at all.

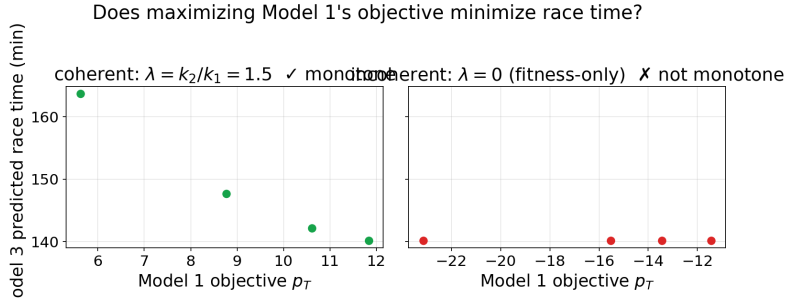


Figure 12: Model 1’s objective against end-to-end predicted race time. Left: $\lambda = k_2/k_1$ gives a strictly monotone relationship. Right: the fitness-only budget does not.

implies a negative energy budget. This is exactly where the objective-coefficient ranging of Section 8 becomes load-bearing: the optimal plan is invariant for $k_2 \in [1.29, 2.95]$, so any $\lambda \in [1.29, 1.75]$ is simultaneously coherent, feasible, and plan-preserving. Setting $\lambda = k_2 = 1.5$ makes the whole system consistent while changing not a single one of the 144 optimal hour variables. The sensitivity analysis is not decorative here; it is what licenses the repair.

9.2 Resolving the circular dependency

The second objection is that the solution order was imposed. Model 1 needs a goal race — the A-race’s distance class fixes the macrocycle length and volume ceiling — while Model 2 needs a fitness trajectory, since its readiness gate is driven by the sustained load Model 1 produces. Each is an input to the other.

Rather than assert an order, we iterate to a fixed point: solve Model 1 under the macrocycle implied by the current A-race class, pass its average build-phase load $\bar{L}$ into Model 2’s readiness projection, re-solve Model 2, and take the distance class of the selected A-race as the next macrocycle. The iteration converged in two steps (Table 11), and it immediately exposed an inconsistency in the sequential presentation: the optimal calendar selects a 70.3 as the season’s A-race, which calls for a 20-week build, whereas the original analysis assumed a 16-week Olympic macrocycle. The self-consistent plan is a 20-week 70.3 macrocycle aimed at Costa Navarino — which is, fittingly, the race the athlete actually contested in 2025.

Iter.	Macrocycle	Weeks	Selected A-race	Season value	Outcome
1	olympic	16	R14 Costa Navarino (70.3)	571.7	re-plan as 70.3
2	70.3	20	R14 Costa Navarino (70.3)	571.7	fixed point

Table 11: Model 1 $\leftrightarrow$ Model 2 iteration. The circular dependency is resolved by convergence rather than by assumption.

Both results are reported as an integration layer rather than folded into the baseline: the shipped default remains $\lambda = 0$, so every figure and validation elsewhere in this report is unaffected and independently checkable.

10 What the Models Recommend

The models were built to be solved, but their purpose is to change decisions. Each item below is traceable to a specific result rather than to coaching opinion.

- **Train more, and much easier.** Realised training over the last 16 weeks was 6.6 h/week with 37% of time at moderate-or-hard intensity; the optimum is 9.0 h/week with 15% — less volume at double the intensity share, the classic anti-polarised pattern. The heuristic benchmark shows the opposite error is equally costly: filling the hours at high intensity breaches the fatigue ceiling in 13 of 16 weeks. The recommendation is more easy volume, not more training.
- **Recovery capacity, not free time, is binding.** RHS ranging shows the objective saturating at 10.5 h/week, beyond which extra hours have zero shadow price because the ATL ceiling binds first. Buying training time is worthless; raising tolerable fatigue (sleep, nutrition, load management) is not, and the ATL duals peak in late build, which is when that investment pays.
- **Build endurance, not peak power.** Below the current 260 W each watt is worth ≈ 6 s of Olympic-distance time, but the marginal value collapses to nearly zero at 260–270 W — with a fixed energy budget, extra power cannot be fuelled. One CTL point is worth 69–84 s up to CTL ≈ 85 .
- **Ride steady, run hard, pace the run evenly.** The optimum rides the whole bike leg at Z3 and splits the run between Z3 and the Z5 cap. The 70.3 validation prices the alternative: going out at 4:53/km and fading to 5:52/km with heart rate dropping from 162 to 145 bpm cost roughly 29 minutes against the even-paced schedule.
- **Fuel at the guideline rate; train the gut only for long course.** The calibrated intake of ≈ 21 kJ/min recovers the standard 75 g carbohydrate/hour. Absorption duals price gut training at ≈ 1.7 min per extra kJ/min of bike capacity at 70.3, and at almost nothing at Olympic distance.
- **Protect the calendar, not the budget.** Six of eight selected races are domestic, and cost ranging shows several stay in even if their value falls to zero. The scarce resource is neither money (0.094 per euro) nor time off (≈ 0 per day) but calendar space: the Hellenic Championship / Costa Navarino recovery conflict prices at 56.4, the largest dual in the model.
- **Two concrete calendar calls.** The early-season Dubai 70.3 is correctly excluded — projected CTL cannot reach the 70.3 readiness threshold by March. And with Costa Navarino as the A-race the macrocycle should be the 20-week 70.3 build, not the 16-week Olympic one, exactly as the fixed-point iteration of Section 9 detected.

11 Discussion and Limitations

Three limitations deserve honesty. First, the weekly Banister recursion is a coarse time step; daily dynamics would shift constants, not the structure of the optimum. Second, the pipeline is solved by decomposition rather than as one joint program: Section 9 replaces the fixed solution order with a fixed-point iteration and shows the objectives can be made coherent, but a single monolithic model co-optimizing macrocycle and calendar would still be a substantially larger MIP — a natural extension. Third, several Model 2/3 parameters (race values, zone speeds, the energy–CTL constant k_E) are constructed from documented assumptions rather than measured; the sensitivity analyses show which conclusions survive parameter perturbation. The relay variant mentioned in the proposal was scoped out in favor of depth (duality, network flows) over breadth.

Against these caveats stand results with practical content: periodization and “steady bike, hard run” emerge as optimal rather than assumed; shadow prices translate into athlete-meaningful quantities (seconds per watt, per kJ, per vacation day); and the analysis exposes which coaching rules are near-optimal (the 80/20 intensity split) and which are not (filling all available hours).

12 Conclusions

The project delivers three coupled optimization models solved on real personal training data, together spanning the course syllabus: LP modeling and Simplex solving (Models 1, 3), duality and sensitivity analysis (shadow prices, RHS ranging, parametric sweeps), integer programming with Branch & Bound and integrality-gap analysis (Model 2), and network flows with total unimodularity (the longest-path reduction). The optimizer independently reproduces core endurance-coaching principles and, through its dual variables, prices the athlete’s true bottlenecks: recovery capacity in training, money in race selection, and aerobic endurance — not peak power — on race day. All code, data assumptions and results are reproducible with a single command from the public repository.¹

References

- [1] E. W. Banister, T. W. Calvert, M. V. Savage, T. Bach, “A systems model of training for athletic performance,” *Australian Journal of Sports Medicine*, 1975.
- [2] G. P. Millet, R. Candau, B. Barbier, T. Busso, J. D. Rouillon, J. C. Chatard, “Modelling the transfers of training effects on performance in elite triathletes,” *International Journal of Sports Medicine*, vol. 23, 2002.
- [3] P. Hellard, M. Avalos, L. Lacoste, F. Barale, J. C. Chatard, G. P. Millet, “Assessing the limitations of the Banister model in monitoring training,” *Journal of Sports Sciences*, vol. 24, 2006.
- [4] I. Mujika, *Tapering and Peaking for Optimal Performance*, Human Kinetics, 2009.
- [5] D. Ceddia, H. Bondell, P. Taylor, “Mathematical Modelling and Optimisation of Athletic Performance: Tapering and Periodisation,” arXiv:2505.20859, 2025.
- [6] G. L. Nemhauser, L. A. Wolsey, *Integer and Combinatorial Optimization*, Wiley, 1988.
- [7] H. P. Williams, *Model Building in Mathematical Programming*, 5th ed., Wiley, 2013.
- [8] C. C. Ribeiro, “Sports scheduling: Problems and applications,” *International Transactions in Operational Research*, vol. 19, 2012.
- [9] M. Trick, “Using Sports Scheduling to Teach Integer Programming,” *INFORMS Transactions on Education*, 2005.
- [10] C. R. Abbiss, P. B. Laursen, “Describing and understanding pacing strategies during athletic competition,” *Sports Medicine*, vol. 38, no. 3, 2008.
- [11] G. P. Millet, V. E. Vleck, D. J. Bentley, “Physiological differences between cycling and running: lessons from triathletes,” *Sports Medicine*, vol. 39, no. 3, 2009.
- [12] S. S. Wu, J. J. Peiffer, J. Brisswalter, K. Nosaka, C. R. Abbiss, “Factors influencing pacing in triathlon,” *Open Access Journal of Sports Medicine*, vol. 5, 2014.
- [13] X. Ni, J. Liu, S. Zhang, P. Ke, “Numerical optimization of pacing strategy in cross-country skiing based on Gauss pseudo-spectral method,” *Scientific Reports*, 2022.
- [14] G. B. Dantzig, *Linear Programming and Extensions*, Princeton University Press, 1963.
- [15] F. S. Hillier, G. J. Lieberman, *Introduction to Operations Research*, 10th ed., McGraw-Hill, 2015.

A Appendix: Implementation

All three models are implemented in Python with PuLP as the modelling layer and CBC as the solver, matching the toolchain used in the course. The listings below are extracted directly from the repository files, so they are the code that produced every number in this report. The full sources, the data pipeline and the 60-test suite are in the repository.

A.1 Model 1 — training-load LP (src/models/model1_training.py)

Variable construction, the Banister dynamics as linear equalities, the constraint block and the objective:

```
42 def build_model(profile: dict, state: dict, rates: pd.DataFrame) -> tuple[pulp.LpProblem, dict]:
43     """Construct the LP. Returns (problem, variables-dict)."""
44     m1 = profile["model1"]
45     mc = profile["macrocycle"]
```

¹<https://github.com/manos1503/triathlon-optimization>

```

46 ban = profile["banister"]
47 T = mc["weeks"]
48 weeks = range(1, T + 1)
49
50 r = {(row.sport, row.bucket): row.trimp_per_hour for row in rates.itertuples()}
51 lam_c = math.exp(-7.0 / ban["tau_ctl_days"])
52 lam_a = math.exp(-7.0 / ban["tau_atl_days"])
53 h_min = profile["constraints"]["weekly_hours_min_per_discipline"]
54 h_max_disc = profile["constraints"].get("weekly_hours_max_per_discipline")
55 h_ceiling = profile["constraints"]["weekly_hours_max"]
56 taper_frac = {int(k): v for k, v in m1["taper_load_fraction"].items()}
57
58 prob = pulp.LpProblem("model1_training_load", pulp.LpMaximize)
59
60 h = pulp.LpVariable.dicts("h", (DISCIPLINES, BUCKETS, weeks), lowBound=0)
61 L = pulp.LpVariable.dicts("L", weeks, lowBound=0)
62 CTL = pulp.LpVariable.dicts("CTL", weeks, lowBound=0)
63 ATL = pulp.LpVariable.dicts("ATL", weeks, lowBound=0)
64
65 # --- dynamics ---
66 for w in weeks:
67     prob += L[w] == pulp.lpSum(r[d, b] * h[d][b][w] for d in DISCIPLINES for b in BUCKETS), f"def_load_w{w}"
68     prev_c = CTL[w - 1] if w > 1 else state["ctl"]
69     prev_a = ATL[w - 1] if w > 1 else state["atl"]
70     prob += CTL[w] == lam_c * prev_c + (1 - lam_c) * L[w] / 7.0, f"def_ctl_w{w}"
71     prob += ATL[w] == lam_a * prev_a + (1 - lam_a) * L[w] / 7.0, f"def_atl_w{w}"
72
73 # --- constraints ---
74 for w in weeks:
75     phase = phase_of(w, mc["phases"])
76     h_max = h_ceiling * m1["hours_multiplier"][phase]
77     total = pulp.lpSum(h[d][b][w] for d in DISCIPLINES for b in BUCKETS)
78
79     prob += total <= h_max, f"hours_cap_w{w}"
80
81     for d in DISCIPLINES:
82         disc_hours = pulp.lpSum(h[d][b][w] for b in BUCKETS)
83         prob += disc_hours >= h_min[d], f"min_{d}_w{w}"
84         if h_max_disc:
85             prob += disc_hours <= h_max_disc[d], f"max_{d}_w{w}"
86
87     prob += (
88         pulp.lpSum(h[d][b][w] for d in DISCIPLINES for b in ("moderate", "hard"))
89         <= m1["polarization_max"] * total
90     ), f"polarization_w{w}"
91
92     prob += ATL[w] <= m1["atl_max"], f"atl_cap_w{w}"
93
94     if w >= 2:
95         prob += L[w] <= (1 + m1["ramp_max"]) * L[w - 1], f"ramp_w{w}"
96
97 # taper: cap vs. average build-phase load
98 build_weeks = range(mc["phases"]["build"]["weeks"][0], mc["phases"]["build"]["weeks"][1] + 1)
99 avg_build = pulp.lpSum(L[w] for w in build_weeks) / len(list(build_weeks))
100 for w, frac in taper_frac.items():
101     prob += L[w] <= frac * avg_build, f"taper_w{w}"
102
103 # --- objective: race-day performance ---
104 prob += ban["k1"] * CTL[T] - ban["k2"] * ATL[T], "race_day_performance"
105
106 return prob, {"h": h, "L": L, "CTL": CTL, "ATL": ATL, "T": T}

```

A.2 Model 2 — race-calendar MIP (src/models/model2_calendar.py)

Binary variables, the two knapsack rows, the monthly caps, the pairwise recovery conflicts, the A-race set-covering row and the readiness fixing:

```

97 def build_model(races: pd.DataFrame, profile: dict,
98                relax: bool = False) -> tuple[pulp.LpProblem, dict]:
99     m2 = profile["model2"]
100     v = race_value(races, m2)
101     cost = races["entry_fee_eur"] + races["travel_cost_eur"]
102     ready = readiness(races, profile)

```

```

103     rec = m2["recovery_weeks"]
104
105     cat = "Continuous" if relax else "Binary"
106     x = {i: pulp.LpVariable(f"x_{races.loc[i, 'id']}", 0, 1, cat) for i in races.index}
107
108     prob = pulp.LpProblem("model2_race_calendar", pulp.LpMaximize)
109     prob += pulp.lpSum(v[i] * x[i] for i in races.index), "season_value"
110
111     prob += pulp.lpSum(cost[i] * x[i] for i in races.index) <= m2["budget_eur"], "budget"
112
113     # second knapsack dimension: vacation days off work
114     if "vacation_days_budget" in m2:
115         prob += (pulp.lpSum(races.loc[i, "vacation_days"] * x[i] for i in races.index)
116                 <= m2["vacation_days_budget"]), "vacation_days"
117
118     months = pd.to_datetime(races["date"]).dt.month
119     for mo in sorted(months.unique()):
120         idx = races.index[months == mo]
121         if len(idx) > 1:
122             prob += pulp.lpSum(x[i] for i in idx) <= m2["max_races_per_month"], f"month_{mo}"
123
124     # recovery spacing (also excludes same-week clashes)
125     for i in races.index:
126         for j in races.index:
127             if i >= j:
128                 continue
129             wi, wj = races.loc[i, "week"], races.loc[j, "week"]
130             earlier, later = (i, j) if wi <= wj else (j, i)
131             gap = abs(int(wj) - int(wi))
132             need = rec[str(races.loc[earlier, "distance_class"])]
133             if gap < need + 1:
134                 prob += x[i] + x[j] <= 1, f"recovery_{races.loc[i, 'id']}_{races.loc[j, 'id']}"
135
136     # at least one championship-quality target (set-covering side constraint)
137     if m2.get("require_a_race", False) and "a_race" in races.columns:
138         a_idx = races.index[races["a_race"] == 1]
139         if len(a_idx):
140             prob += pulp.lpSum(x[i] for i in a_idx) >= 1, "a_race_requirement"
141
142     # fitness-readiness pre-filter (Model 1 link)
143     for i in races.index:
144         if not ready[i]:
145             prob += x[i] == 0, f"not_ready_{races.loc[i, 'id']}"
146
147     return prob, {"x": x, "value": v, "cost": cost, "ready": ready}

```

A.3 Model 3 — pacing LP (src/models/model3_pacing.py)

The zone-discretized time variables, distance coverage with the bike→run coupling, the energy budget (fitness-only or form-based) and the intensity caps:

```

90     cycling power balance, with nothing fitted to race results. G = 0 (the
91     default, a flat reference course) leaves the speeds unchanged.
92     """
93     m3 = profile["model3"]
94     thr = profile["thresholds"]
95     ftp = ftp_watts if ftp_watts is not None else thr["bike_ftp_watts"]
96
97     v_swim_thr = _pace_to_kmh(m3["swim_threshold_pace_per_100m"], 100) / 60.0 # km/min
98     v_run_thr = _pace_to_kmh(thr["run_threshold_pace"], 1000) / 60.0
99     gf = grade_factor(profile, gradient_m_per_km, ftp)
100     v_bike_ftp_ref = m3["bike_speed_at_ftp_kmh"] * gf / 60.0
101     ftp_ref = thr["bike_ftp_watts"]
102
103     rows = []
104     for z in ZONES:
105         cost = m3["intensity_cost_factor"][z]
106
107         v = v_swim_thr * m3["swim_speed_fraction"][z]
108         rows.append(("swim", z, v, m3["swim_energy_cost_kj_per_km"] * cost * v))
109
110         p = ftp * m3["power_fraction"][z]
111         # speed scales with cube root of power (aero drag); reference 35 km/h at profile FTP
112         v = v_bike_ftp_ref * (p / ftp_ref) ** (1.0 / 3.0)

```

```

113     e = p / m3["bike_metabolic_efficiency"] * 60.0 / 1000.0 # kJ/min
114     rows.append(("bike", z, v, e))
115
116     v = v_run_thr * m3["run_speed_fraction"][z]
117     rows.append(("run", z, v, m3["run_energy_cost_kj_per_km"] * cost * v))
118
119     return pd.DataFrame(rows, columns=["leg", "zone", "speed_km_min", "energy_kj_min"])
120
121
122 def energy_budget(profile: dict, ctl: float, atl: float = 0.0,
123                  lam: float | None = None) -> float:
124     """Race-day energy budget.
125
126     Fitness-only (default, lam = 0): E = k_E * CTL
127     Form-based (lam > 0): E = E_base + k_E * (CTL - lam * ATL)
128
129     The form-based version is what makes the pipeline coherent: it is an
130     increasing affine function of Model 1's objective p_T = k1*CTL - k2*ATL
131     exactly when lam = k2/k1, so maximizing p_T is equivalent to maximizing
132     race-day energy, hence to minimizing finishing time (Model 3's objective
133     is non-increasing in E). See src/analysis/coherence.py.
134     """
135     m3 = profile["model3"]
136     lam = m3.get("fatigue_lambda", 0.0) if lam is None else lam
137     k_e = m3["energy_budget_kj_per_ctl"]
138     if lam == 0.0:
139         return k_e * ctl
140     return m3.get("energy_base_kj", 0.0) + k_e * (ctl - lam * atl)
141
142
143 def build_model(profile: dict, ctl_race_day: float,
144                ftp_watts: float | None = None,
145                fueling_kj_min: float | None = None,
146                fueling_decision: bool = False,
147                atl_race_day: float = 0.0,
148                fatigue_lambda: float | None = None,
149                gradient_m_per_km: float = 0.0) -> tuple[pulp.LpProblem, dict]:

```

B Appendix: Program Output

Verbatim console output of the three solvers, produced by running the commands shown. Everything reported in the body of this document can be regenerated with a single command, `python -m src.run_all`, which additionally rebuilds all figures and result tables.

```

$ python -m src.models.run_model1
[model1] status=Optimal objective p_T=-11.44
[model1] start: CTL=78.1 ATL=91.46
week load hours ctl atl tsb
1 845.5 10.8 84.7 110.0 -13.4
2 770.0 8.6 88.5 110.0 -25.3
3 770.0 8.6 91.8 110.0 -21.5
4 770.0 10.6 94.6 110.0 -18.2
5 770.0 10.8 97.0 110.0 -15.4
6 770.0 10.8 99.0 110.0 -13.0
7 770.0 10.7 100.7 110.0 -11.0
8 770.0 9.9 102.1 110.0 -9.3
9 770.0 11.0 103.3 110.0 -7.9
10 770.0 9.5 104.3 110.0 -6.7
11 770.0 9.9 105.2 110.0 -5.7
12 770.0 9.1 105.9 110.0 -4.8
13 770.0 9.8 106.6 110.0 -4.1
14 302.5 4.5 96.8 67.8 -3.4
15 302.5 4.5 88.6 52.3 29.1
16 302.5 4.5 81.6 46.5 36.4
[model1] avg weekly hours 9.0, race-day CTL=81.6 ATL=46.5 TSB=35.1

$ python -m src.models.run_model2
[model2] MIP z=571.7 LP relaxation z=600.5 gap 5.04%
[model2] budget 2250/2500 EUR, vacation days 11/12, 8 races selected, 1 excluded by readiness

selected calendar:
id name date distance_class cost value
R01 Attica Sprint (season opener) 2027-03-14 sprint 45 45.106667

```

```

R02 TRIMORE Rethymno Triathlon 2027-04-18 olympic 290 83.838333
R03 TRIMORE Syros Olympic 2027-05-09 olympic 210 56.638333
R05 Epidavros Legacy Standard 2027-06-06 olympic 60 69.120000
R07 Thessaloniki Sprint 2027-06-27 sprint 160 46.636667
R09 Nafplio Olympic 2027-07-18 olympic 135 47.950000
R11 Ironman 70.3 Budapest 2027-08-22 70.3 770 104.727513
R14 Ironman 70.3 Greece Costa Navarino 2027-09-26 70.3 580 117.652085

```

fractional in LP relaxation (rounded off by B&B):

```

id name x
R08 Ironman 70.3 Nice 0.1828
R09 Nafplio Olympic 0.8172
R10 London T100 Age-Group 0.1828
R11 Ironman 70.3 Budapest 0.5000
R12 Copenhagen Olympic 0.5000
R13 Hellenic Champs Olympic 0.5000
R14 Ironman 70.3 Greece Costa Navarino 0.5000
R16 MTRI Valencia Sprint 0.5000

```

LP-relaxation shadow prices:

```

budget: 0.0936
vacation_days: -0.0000

```

```

$ python -m src.models.run_model3
[model3] CTL=81.6 -> E_tot=9389 kJ
[model3] total 140.4 min (swim 26.1 / bike 71.0 / run 39.8)
leg zone minutes km kj
swim Z3 15.65 0.86 1060.94
swim Z4 6.02 0.36 473.26
swim Z5 4.41 0.28 386.43
bike Z3 71.02 40.00 4154.81
run Z3 19.91 4.62 1419.92
run Z5 19.91 5.38 1893.23

```

shadow prices:

```

constraint shadow_price meaning
dist_swim 26.4183 marginal min per extra swim km
dist_bike 2.5085 marginal min per extra bike km
dist_run 6.3207 marginal min per extra run km
energy_budget -0.0071 min saved per extra kJ of energy (fitness value)
hard_cap_swim -0.0505 min saved per unit relaxed swim intensity cap
hard_cap_run -0.0764 min saved per unit relaxed run intensity cap

```

C Appendix: Where Each Result Comes From

D Appendix: the Simplex Method on a Reduced Instance

CBC solves every LP in this project with the (dual) Simplex algorithm. To make the method visible rather than implicit, Model 3 is reduced to two variables: the run leg with two zones,

$$\min t_3 + t_5 \quad \text{s.t.} \quad 0.2317 t_3 + 0.2704 t_5 \geq 10 \quad (\text{distance}), \quad 71.3 t_3 + 95.1 t_5 \leq 3300 \quad (\text{energy}), \quad t \geq 0,$$

with speeds (km/min) and energy rates (kJ/min) taken from the athlete's Z3/Z5 run parameters and a 3300 kJ energy share for the leg.

The feasible region (Fig. 13) has vertices $A = (43.16, 0)$ (all-Z3, time 43.16, energy slack 223 kJ), $B = (21.30, 18.73)$ (both constraints binding, time 40.03) and the infeasible all-Z5 corner $C = (0, 34.70)$. In standard form the problem gains a surplus variable s_1 (distance), a slack s_2 (energy) and one artificial variable for Phase I. Phase I drives the artificial variable out, landing at vertex A with basis $\{t_3, s_2\}$; in Phase II the reduced cost of t_5 is negative, so t_5 enters and s_2 leaves — the ratio test blocks at the energy constraint — and the basis $\{t_3, t_5\}$ at B is optimal, both reduced costs being nonnegative. The path $A \rightarrow B$ is exactly one Simplex pivot; CBC reports the same optimum, $t = 40.03$ minutes. The duals at B solve $m^\top y = c$ with m the binding-constraint matrix: distance is priced at 8.64 min/km and energy at -0.014 min/kJ. Both are larger in magnitude than the full model's run-distance and energy duals (6.32 and -0.0071), as they should be: the toy has no other zones or legs to absorb a perturbation, so each marginal km or kJ hits harder.

Result	Produced by	Output
Data pipeline	src/pipeline/run_pipeline.py	fitness_state.yaml, trimp_rates.csv
Model 1 solve (Fig. 2, 3)	src/models/run_model1.py	model1_plan.csv, model1_trajectory.csv
RHS ranging (Fig. 4)	src/analysis/rhs_ranging.py	model1_rhs_ranging.csv
Shadow prices	src/analysis/shadow_prices.py	model1_shadow_prices.csv
Alternative optima	src/analysis/alt_optima.py	model1_alt_optima_summary.csv
Heuristic benchmark	src/analysis/heuristic_benchmark.py	model1_heuristic_benchmark.csv
Actual vs optimal (Fig. 5)	src/analysis/actual_vs_optimal.py	model1_actual_vs_optimal.csv
Model 2 solve (Fig. 6)	src/models/run_model2.py	model2_selection.csv, model2_lp_duals.csv
Budget sweep (Fig. 7)	src/analysis/budget_sweep.py	model2_budget_sweep.csv
Clique cuts	src/analysis/clique_cuts.py	model2_clique_cuts.csv
Network flow	src/analysis/network_flow.py	model2_network_flow.csv
Model 3 solve (Fig. 8)	src/models/run_model3.py	model3_pacing_plan.csv, model3_duals.csv
FTP/CTL sweeps (Fig. 9)	src/analysis/ftp_sweep.py	model3_ftp_sweep.csv
Race validation (Tab. 7)	src/analysis/race_validation.py	model3_race_validation.csv
Distance scenarios (Fig. 10)	src/analysis/distance_scenarios.py	model1_distance_scenarios.csv
Cost ranging (Tab. 9)	src/analysis/cost_ranging.py	model1/2_cost_ranging.csv
k_2 sensitivity (Fig. 11)	src/analysis/k2_sensitivity.py	model1_k2_sensitivity.csv
Coherence (Fig. 12)	src/analysis/coherence.py	coherence*.csv
Fixed point (Tab. 11)	src/analysis/pipeline_loop.py	pipeline_loop.csv
Simplex example (Fig. 13)	src/analysis/simplex_toy.py	simplex_toy.png

Table 12: Mapping from every reported result to the module that generates it.

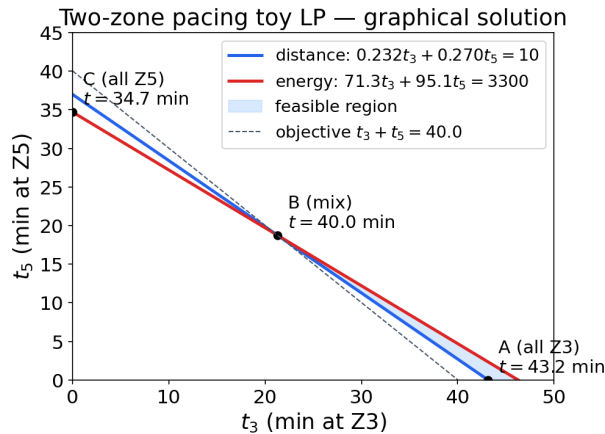


Figure 13: Feasible region and vertices of the reduced pacing LP. The optimum is vertex B , the intersection of the two constraints.